



412. Fizz Buzz

Description:

- ✓ Given an integer n, return a string array answer (1-indexed) where:
- ✓ `answer[i] == "FizzBuzz"` if i is divisible by 3 and 5.
- ✓ `answer[i] == "Fizz"` if i is divisible by 3.
- ✓ `answer[i] == "Buzz"` if i is divisible by 5.
- ✓ `answer[i] == i` (as a string) if none of the above conditions are true.

Example 1:

Input: n = 3

Output: ["1","2","Fizz"]

Example 2:

Input: n = 5

Output: ["1","2","Fizz","4","Buzz"]

Example 3:

Input: n = 15

Output:

["1","2","Fizz","4","Buzz","Fizz","7","8","Fizz","Buzz","11","Fizz","13","14","FizzBuzz"]

Constraints:

- a) $1 \leq n \leq 104$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google 5

Amazon 5

IBM 2

Cisco 2

JPMorgan 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Facebook 3

Apple 3

Microsoft 2

Uber 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

TikTok 6

Bloomberg 4

LinkedIn 2

Oracle 2

Expedia 2

412. Fizz Buzz - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    public List<String> fizzBuzz(int n) {

        List<String> result = new ArrayList<>();

        for (int i = 1; i <= n; i++) {

            boolean divBy3 = i % 3 == 0;
            boolean divBy5 = i % 5 == 0;
```



```
String currStr = "";

if (divBy3) {
    currStr += "Fizz";
}

if (divBy5) {
    currStr += "Buzz";
}

if (currStr.isEmpty()) {
    currStr += String.valueOf(i);
}

result.add(currStr);
}

return result;
}
}
```



12. Integer to Roman

Description:

- ✓ Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

- ✓ For example, 2 is written as II in Roman numeral, just two one's added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.
- ✓ Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:
 - ✓ I can be placed before V (5) and X (10) to make 4 and 9.
 - ✓ X can be placed before L (50) and C (100) to make 40 and 90.
 - ✓ C can be placed before D (500) and M (1000) to make 400 and 900.
 - ✓ **Given an integer, convert it to a roman numeral.**

Example 1:

Input: num = 3

Output: "III"

Explanation: 3 is represented as 3 ones.

Example 2:

Input: num = 58

Output: "LVIII"

Explanation: L = 50, V = 5, III = 3.

Example 3:

Input: num = 1994

Output: "MCMXCIV"

Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.

Constraints:

a) $1 \leq \text{num} \leq 3999$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 15

Google 4

Facebook 4

Adobe 3

Oracle 3

Citadel 3

Microsoft 2

Bloomberg 2

Apple 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

LinkedIn 3

Capital One 3

Morgan Stanley 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

BlackRock 3

Uber 2

TikTok 2

Twitter



12. Integer to Roman - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    private static final int[] values = { 1000, 900, 500, 400, 100, 90, 50, 40, 10, 9,
    5, 4, 1 };
    private static final String[] symbols = { "M", "CM", "D", "CD", "C", "XC", "L",
    "XL", "X", "IX", "V", "IV", "I" };

    public String intToRoman(int num) {

        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < values.length && num > 0; i++) {

            while (values[i] <= num) {
                num -= values[i];
                sb.append(symbols[i]);
            }

        }

        return sb.toString();
    }
}
```



13. Roman to Integer

Description:

- ✓ Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

- ✓ For example, 2 is written as II in Roman numeral, just two one's added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.
- ✓ Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:
 - ✓ I can be placed before V (5) and X (10) to make 4 and 9.
 - ✓ X can be placed before L (50) and C (100) to make 40 and 90.
 - ✓ C can be placed before D (500) and M (1000) to make 400 and 900.
 - ✓ **Given a roman numeral, convert it to an integer.**

Example 1:

Input: str = "III"

Output: 3

Explanation: III = 3.

Example 2:

Input: str = "LVIII"

Output: 58

Explanation: L = 50, V = 5, III = 3.

Example 3:

Input: str = "MCMXCIV"

Output: 1994

Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.

Constraints:

- a) $1 \leq \text{str.length} \leq 15$
- b) str contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
- c) It is guaranteed that str is a valid roman numeral in the range [1, 3999].

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 30

Adobe 25

Apple 21

Google 20

Microsoft 13

Facebook 7

Bloomberg 6

Yahoo 3

Oracle 3

tcs 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Uber 4

TikTok 3

Intel 2

Qualtrics 2

Morgan Stanley 2

Yandex 2

Accenture 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Qualcomm 5

LinkedIn 4

JPMorgan 3

HBO 3

VMware 2

Goldman Sachs 2

Flipkart 2

13. Roman to Integer - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    static Map<String, Integer> mymap = new HashMap<>();

    static {
        mymap.put("M", 1000);
        mymap.put("D", 500);
        mymap.put("C", 100);
        mymap.put("L", 50);
        mymap.put("X", 10);
        mymap.put("V", 5);
        mymap.put("I", 1);
    }

    public int romanToInt(String str) {

        int n = str.length();
        int sum = 0;
        int i = 0;
        while (i < n) {
```



```
String currSymbol = str.substring(i, i + 1);
int currValue = mymap.get(currSymbol);

int nextValue = 0;

if (i + 1 < n) {
    String nextSymbol = str.substring(i + 1, i + 2);
    nextValue = mymap.get(nextSymbol);
}

if (currValue < nextValue) {
    sum += (nextValue - currValue);
    i += 2;
}
else {
    sum += currValue;
    i += 1;
}

return sum;
}
```



705. Design HashSet

Description:

- ✓ Design a HashSet without using any built-in hash table libraries.
- ✓ Implement MyHashSet class:
 - a) **void add(key) :**
Inserts the value key into the HashSet.
 - b) **bool contains(key):**
Returns whether the value key exists in the HashSet or not.
 - c) **void remove(key):**
Removes the value key in the HashSet.
If key does not exist in the HashSet, do nothing.

Example 1:

Input

```
["MyHashSet", "add", "add", "contains", "contains", "add", "contains", "remove",  
"contains"]  
[[], [1], [2], [1], [3], [2], [2], [2], [2]]
```

Output

```
[null, null, null, true, false, null, true, null, false]
```

Explanation

```
MyHashSet myHashSet = new MyHashSet();
```

```
myHashSet.add(1);    // set = [1]  
myHashSet.add(2);    // set = [1, 2]  
myHashSet.contains(1); // return True  
myHashSet.contains(3); // return False, (not found)  
myHashSet.add(2);    // set = [1, 2]  
myHashSet.contains(2); // return True  
myHashSet.remove(2);  // set = [1]  
myHashSet.contains(2); // return False, (already removed)
```

Constraints:

- a) $0 \leq \text{key} \leq 106$
- b) At most 104 calls will be made to add, remove, and contains.

Companies

6 months - 1 year 1 year - 2 years

Amazon 4

Microsoft 4

Apple 2

Companies

6 months - 1 year 1 year - 2 years

Google 2

Wix

705. Design HashSet - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyHashSet {

    int bucketSize;
    List<LinkedList<Integer>> mytable;

    public MyHashSet() {
        this.bucketSize = 7;
        this.mytable = new ArrayList<LinkedList<Integer>>(bucketSize);

        for(int i=0;i<bucketSize;i++) {
            mytable.add(new LinkedList<Integer>());
        }
    }
}
```



```
public void add(int key) {
    int index = myhash(key);
    if( !contains(key)) {
        mytable.get(index).add(key);
    }
}

public void remove(int key) {
    int index = myhash(key);
    mytable.get(index).remove((Integer)key);
}

public boolean contains(int key) {
    int index = myhash(key);
    return mytable.get(index).contains(key);
}

public int myhash(Integer key) {
    int hash = key.hashCode() % this.bucketSize;
    return hash;
}
}
```



706. Design HashMap

Description:

- ✓ Design a HashMap without using any built-in hash table libraries.
- ✓ Implement the MyHashMap class:
 - a) **MyHashMap():**
Initializes the object with an empty map.
 - b) **void put(int key, int value):**
Inserts a (key, value) pair into the HashMap.
If the key already exists in the map, update the corresponding value.
 - c) **int get(int key):**
Returns the value to which the specified key is mapped,
or -1 if this map contains no mapping for the key.
 - d) **void remove(key):**
Removes the key and its corresponding value if the map contains the
mapping for the key.

Example 1:

Input

["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]

[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]

Output

[null, null, null, 1, -1, null, 1, null, -1]

Explanation

MyHashMap myHashMap = new MyHashMap();

myHashMap.put(1, 1); // The map is now [[1,1]]

myHashMap.put(2, 2); // The map is now [[1,1], [2,2]]

myHashMap.get(1); // return 1, The map is now [[1,1], [2,2]]

myHashMap.get(3); // return -1 (i.e., not found), The map is now [[1,1], [2,2]]

myHashMap.put(2, 1); // The map is now [[1,1], [2,1]] (i.e., update the existing value)

myHashMap.get(2); // return 1, The map is now [[1,1], [2,1]]

myHashMap.remove(2); // remove the mapping for 2, The map is now [[1,1]]

myHashMap.get(2); // return -1 (i.e., not found), The map is now [[1,1]]

Constraints:

- a) $0 \leq \text{key}, \text{value} \leq 106$
- b) At most 104 calls will be made to put, get, and remove.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon 5 Microsoft 4 Apple 4

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

VMware 3 Shopee 3 Twitter 2 LinkedIn 2 Goldman Sachs 2

Facebook 2 Google 2 Oracle 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Salesforce 5 ServiceNow 4 Adobe 3 Walmart Global Tech 3

Bloomberg 3 Uber 2 ByteDance 2 Zynga 2



706. Design HashMap - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

class MyEntry<K, V> {
    public K key;
    public V value;

    public MyEntry(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public String toString() {
        return key + " - " + value;
    }
}

class MyHashMap {

    private int bucketSize;
    List<LinkedList<MyEntry<Integer, Integer>>> mytable;

    public MyHashMap() {
        this.bucketSize = 7;
        this.mytable = new ArrayList<LinkedList<MyEntry<Integer,
        Integer>>>(bucketSize);

        for (int i = 0; i < bucketSize; i++) {
            mytable.add(new LinkedList<MyEntry<Integer, Integer>>());
        }
    }
}
```



```
public void put(int mykey, int myvalue) {
```

```
    int index = myhash(mykey);
    LinkedList<MyEntry<Integer, Integer>> mylinkedList =
    mytable.get(index);
    boolean found = false;
    for (MyEntry<Integer, Integer> myentry : mylinkedList) {
        if (myentry.key.equals(mykey)) {
            myentry.value = myvalue;
            found = true;
        }
    }
    if (!found) {
        mylinkedList.add(new MyEntry<Integer, Integer>(mykey, myvalue));
    }
}
```

```
public void remove(int mykey) {
```

```
    int index = myhash(mykey);

    LinkedList<MyEntry<Integer, Integer>> mylinkedList =
    mytable.get(index);

    for (MyEntry<Integer, Integer> myentry : mylinkedList) {
        if (myentry.key.equals(mykey)) {
            mylinkedList.remove(myentry);
            break;
        }
    }
}
```



```
public int get(int mykey) {
    int index = myhash(mykey);
    LinkedList<MyEntry<Integer, Integer>> mylinkedList =
        mytable.get(index);

    for (MyEntry<Integer, Integer> myentry : mylinkedList) {
        if (myentry.key.equals(mykey))
            return myentry.value;
    }
    return -1;
}

public int myhash(Integer key) {
    int hash = key.hashCode() % this.bucketSize;
    return hash;
}
}
```